



Complete Study Notes & Technical Reference
Everything You Need to Understand Your Own Library

MSc Thesis Project | Open Source Python Library
`pip install shaaha` | pypi.org/project/shaaha

Table of Contents

#	Chapter	Description
1.	What is Shaaha?	The big picture
2.	Core Concepts & Definitions	Every term explained
3.	How Python Imports Work	The foundation
4.	PEP 302 — Import Hooks	The key mechanism
5.	File-by-File Explanation	Every file in the project
6.	The 3 Design Patterns Used	Proxy, Strategy, Registry
7.	How Routing Works	Step-by-step decision making
8.	Supported Domains & Backends	The full registry
9.	What the AI Element Is	Context-aware intelligence
10.	How to Extend Shaaha	Adding new backends
11.	MSc Thesis Key Points	What to write in your thesis
12.	Quick Reference Card	Cheat sheet

Chapter 1 — What is Shaaha?

Shaaha is a Python library that acts as an **Intelligent Meta-Dispatcher**. Instead of importing numpy, pandas, or torch directly, you import shaaha — and Shaaha automatically selects the best available library for your hardware and environment. It is invisible to the user but powerful under the hood.

The Problem It Solves

Every Python developer who works in data science or machine learning has faced this:

```
try: import cupy as np # GPU version except ImportError: import numpy as np # CPU
fallback
```

This pattern is ugly, repetitive, and breaks on different machines. Shaaha eliminates it entirely. You write one clean import and Shaaha handles the rest silently.

The Solution

```
import shaaha.math as sm # Shaaha picks cupy (GPU) or numpy (CPU) for you
arr = sm.array([1, 2, 3]) # Works the same either way
```

Shaaha also handles:

- **Data scale decisions** — uses polars for large datasets, pandas for small ones
- **Hardware detection** — detects CUDA GPU, Apple MPS, or CPU automatically
- **Dependency fallback** — if the best library is not installed, falls back gracefully
- **Unified API** — your code stays the same even when the backend changes

Chapter 2 — Core Concepts & Definitions

Every term used in the Shaaha project, explained in plain English:

Meta-Dispatcher

A system that sits between the user and multiple libraries, deciding at runtime which library to use. 'Meta' means it operates at a higher level than the libraries themselves. Shaaha is a meta-dispatcher because it dispatches (routes) your request to the right library.

Import Hook

A piece of Python code that intercepts the import statement before Python looks for a physical file. When you write 'import shaaha.math', Python checks `sys.meta_path` first and Shaaha's hook captures that call.

`sys.meta_path`

A Python built-in list that holds 'Finder' objects. When Python sees any import statement, it walks through this list asking each Finder if it knows where the module is. Shaaha inserts itself at position 0 — the very front of this list.

Finder (MetaPathFinder)

A class with a `find_spec()` method. Python calls `find_spec()` with the module name. If the Finder recognises the module, it returns a `ModuleSpec`. If not, it returns `None` and Python tries the next Finder in `sys.meta_path`.

Loader

A class that actually builds the module object. After a Finder says 'yes I know this module', Python calls the Loader's `exec_module()` method to create and populate the module.

ModuleSpec

A data structure (introduced in PEP 451) that holds metadata about a module: its name, which loader should load it, where it comes from, and whether it is a package. Shaaha creates a `ModuleSpec` pointing to its custom Loader.

ProxyModule

A Python object that behaves exactly like a module but has no real content of its own. Every attribute access (like `shaaha.math.array`) is forwarded to the real backend. The proxy is what makes the API transparent and unified.

Lazy Loading

A technique where expensive operations (like importing a large library) are deferred until the moment they are actually needed. Shaaha does not import `numpy` or `polars` when you write 'import shaaha.math' — it only imports when you access the first attribute.

Backend

The actual library that does the real work. For `shaaha.math`, the backend could be `numpy`, `cupy`, or `jax`. The backend is hidden from the user — they only see `shaaha`.

Domain

A category of functionality in Shaaha. 'math', 'dataframe', 'ml', 'image' are all domains. Each domain has multiple possible backends registered in the Registry.

Registry

The knowledge base of Shaaha. A Python dictionary that maps each domain to a list of Backend objects, each with a weight (priority score) and requirements.

Router

The decision engine of Shaaha. Given a domain, the Router checks the Registry, probes the environment, and selects the highest-scoring available backend.

Weight / Score

A number assigned to each backend indicating how preferred it is. Higher weight = higher priority. cupy gets weight 95 because it is fastest on GPU. numpy gets 70 as a reliable universal fallback.

Environment Probe

A lightweight scan of the system done once at startup. Checks: Is CUDA available? Is MPS (Apple GPU) available? How many CPU cores? Which libraries are installed?

PEP 302

Python Enhancement Proposal 302, titled 'New Import Hooks'. Published in 2003. This document defined the Finder and Loader protocol that Shaaha uses. It is the official specification for how import hooks work.

PEP 451

Python Enhancement Proposal 451, titled 'A ModuleSpec Type for the Import System'. Published in 2013. Modernised PEP 302 by introducing ModuleSpec. Shaaha uses both the PEP 302 interface (find_spec) and PEP 451 (ModuleSpec, exec_module).

PEP 621

Python Enhancement Proposal 621, titled 'Storing project metadata in pyproject.toml'. Defines the standard format for Python package configuration. Shaaha's pyproject.toml follows this standard, which is why it can be published to PyPI.

PyPI

Python Package Index — the official repository of Python packages at pypi.org. When you run 'pip install shaaha', pip downloads Shaaha from PyPI.

CUDA

Compute Unified Device Architecture — NVIDIA's platform for GPU computing. If CUDA is available, Shaaha prefers GPU-accelerated backends like cupy or torch.

MPS

Metal Performance Shaders — Apple's GPU acceleration framework for M1/M2/M3 chips. Shaaha detects MPS availability and boosts Apple-optimised backends when running on a Mac with Apple Silicon.

__getattr__

A Python special method called when attribute lookup fails normally. Shaaha's ProxyModule overrides `__getattr__` so that any attribute access triggers backend resolution and then delegates to the real backend's attribute.

types.ModuleType

The base class for all Python modules. Shaaha's ProxyModule inherits from this class so that it looks and behaves exactly like a real module to Python and IDEs.

Singleton Pattern

A design pattern where only one instance of a class exists. Shaaha's Router uses a metaclass (`_RouterMeta`) to ensure there is always exactly one Router with one shared cache, no matter how many times it is referenced.

Dependency Hell

The common problem where different projects require different versions of the same library, or where installing one library breaks another. Shaaha reduces dependency hell because it has ZERO required

dependencies and works with whatever you have.

Chapter 3 — How Python Imports Work

Before you can understand Shaaha, you need to understand what happens when Python sees an import statement. This is the foundation of the entire project.

Step-by-Step: What Python Does with 'import X'

Step	What Python Does	Where Shaaha Intervenes
1	Checks sys.modules cache — if already imported, returns it immediately	No
2	Walks through sys.meta_path asking each Finder's find_spec() method	YES — ShaahafFinder is here
3	If a Finder returns a ModuleSpec, Python calls the Loader	YES — ShaahaaLoader runs
4	Loader calls exec_module() to populate the module	YES — ProxyModule is inserted
5	Module is added to sys.modules and returned to the user	No — standard Python

The key insight: **Python always checks sys.meta_path before looking for any file.** This is the hook Shaaha uses. By placing ShaahafFinder at position 0 in sys.meta_path, Shaaha gets first look at every import statement in the entire program.

sys.modules — The Import Cache

Python keeps every imported module in a dictionary called sys.modules. The key is the module name (e.g. 'shaaha.math') and the value is the module object. Once a module is in sys.modules, Python never imports it again — it returns the cached version. This is why Shaaha's lazy loading is efficient: the ProxyModule is cached in sys.modules, and the backend is cached inside the ProxyModule.

Chapter 4 — PEP 302: Import Hooks (The Key Mechanism)

PEP 302 is the Python Enhancement Proposal that made Shaaha possible. Published in 2003, it defined a protocol for intercepting Python's import system.

The Two Classes You Must Implement

Class	Method to Implement	What It Does
MetaPathFinder	find_spec(fullname, path, target)	Decides whether to handle an import. Returns ModuleSpec
Loader	exec_module(module)	Builds the module content. Called by Python after find_spec

Shaaha's Implementation

```
class ShaahafFinder(importlib.abc.MetaPathFinder):
    def find_spec(self, fullname, path, target=None):
        # Only handle 'shaaha.something' imports if not
        # already found
        if fullname.startswith('shaaha.'):
            domain = fullname.split('.')[1]
            # e.g. 'math'
            loader = ShaahaaLoader(fullname, domain)
            return ModuleSpec(fullname, loader)
        # Tell Python: I handle this
```

```
class ShaahaaLoader(importlib.abc.Loader):
    def exec_module(self, module):
        # Replace the empty module with our ProxyModule
        proxy = ProxyModule(self.domain, Router)
        module.__class__ = proxy.__class__
        module.__dict__.update(proxy.__dict__)
```

When the user writes `import shaaha.math`, Python calls `ShaahafFinder.find_spec('shaaha.math', ...)`. The finder sees it starts with 'shaaha.' and returns a `ModuleSpec`. Python then calls `ShaahaaLoader.exec_module()` which inserts a `ProxyModule`. The whole process takes microseconds.

Chapter 5 — File-by-File Explanation

Here is every file in the Shaaha project explained in full detail:

`__init__.py` — The Entry Point

This is the first file Python runs when you write 'import shaaha'. It does three things: 1. Creates an instance of ShaahafFinder 2. Inserts it at `sys.meta_path[0]` (the front of the queue) 3. Exposes the public API: `configure()`, `status()`, `available_backends()` It also calls `Environment.probe()` once to scan the hardware. Everything else is lazy.

```
import sys from shaaha.importer import ShaahafFinder _finder = ShaahafFinder()
sys.meta_path.insert(0, _finder) # Register the hook
```

`importer.py` — The Gatekeeper

Contains ShaahafFinder and ShaahaaLoader. ShaahafFinder intercepts any 'shaaha.*' import. It skips internal modules (importer, router, registry etc.) so those load normally. ShaahaaLoader builds the ProxyModule and injects it into the module object that Python provides.

```
def find_spec(self, fullname, path, target=None): if not
fullname.startswith('shaaha.'): return None domain = fullname.split('.')[1] if domain
in _internals: return None # let real files load return ModuleSpec(fullname,
ShaahaaLoader(fullname, domain))
```

`environment.py` — The Hardware Detective

Probes the system ONCE at startup. Uses `importlib.util.find_spec()` to check which libraries are installed WITHOUT importing them. Checks for CUDA using `torch.cuda.is_available()` only if torch is installed. Returns an Environment dataclass with all findings.

```
def _is_importable(name): return importlib.util.find_spec(name) is not None # Check
CUDA without importing torch unnecessarily if _is_importable('torch'): import torch
env.cuda_available = torch.cuda.is_available()
```

`registry.py` — The Knowledge Base

A Python dictionary mapping domain names to lists of Backend objects. Each Backend has: name (pip name), import_name (Python import), weight (priority 0-100), gpu_preferred flag, requires_cuda flag, and tags. The Registry class provides query methods. Community contributors add new backends here.

```
REGISTRY = { 'math': [ Backend('cupy', 'cupy', weight=95, gpu_preferred=True,
requires_cuda=True), Backend('numpy', 'numpy', weight=70), ], }
```

`router.py` — The Brain

The `Router.resolve(domain)` method is the core of Shaaha's intelligence. It: 1) Gets all registered backends for the domain 2) Filters out backends that require CUDA/MPS if hardware is absent 3) Checks each remaining backend is importable 4) Boosts GPU backends by +20 if GPU is available and

prefer_gpu=True 5) Returns the highest-scoring backend 6) Caches the result so routing only happens once per domain.

```
effective = b.weight if b.gpu_preferred and cuda and prefer_gpu: effective += 20 # GPU
boost scored.append((effective, b)) scored.sort(reverse=True) return scored[0][1] #
Winner
```

proxy.py — The Impersonator

ProxyModule inherits from types.ModuleType so it IS a real Python module. `__getattr__` is overridden to intercept every attribute access. On first access, it calls `Router.resolve()` to get the real backend, then caches it. All subsequent accesses go directly to the cached backend via `getattr(backend, name)`. `__dir__` delegates to the backend so IDE autocomplete works.

```
def __getattr__(self, name): backend = self._resolve() # lazy, cached return
getattr(backend, name) # forward to real library def _resolve(self): if self._backend
is None: self._backend = router.resolve(self.domain) return self._backend
```

installer.py — The Auto-Installer

Optional Phase 3 feature. When `auto_install=True` and a backend is missing, `Installer.install()` prompts the user and runs `'pip install '` using `subprocess`. Uses `sys.executable` to ensure the correct Python environment's pip is used. Calls `importlib.invalidate_caches()` after install to refresh Python's module finder.

```
result = subprocess.run( [sys.executable, '-m', 'pip', 'install', package],
capture_output=True ) if result.returncode == 0: importlib.invalidate_caches() return
True
```

wrappers/image_pillow.py — The Normaliser

When a backend has a different API from the standard, a wrapper normalises it. `image_pillow.py` wraps Pillow's API into Shaaha's standard image interface. This means `shaaha.image.open()` works whether OpenCV or Pillow is the backend. Wrappers are the key to API unification.

```
def open(path): # Shaaha standard name from PIL import Image return
Image.open(str(path)) def resize(image, w, h): # Shaaha standard signature return
image.resize((w, h), Image.LANCZOS)
```

Chapter 6 — The 3 Design Patterns Used

Shaaha is built on three classic software engineering design patterns. These are important for your MSc thesis.

Pattern	Where Used in Shaaha	What It Does
Proxy Pattern	proxy.py — ProxyModule	Wraps the real backend and forwards all calls transparently
Strategy Pattern	router.py — Router	Selects the best algorithm (backend) at runtime based on context
Registry Pattern	registry.py — REGISTRY dict	Central store of available implementations, looked up by key (context)

Proxy Pattern — Explained

The Proxy pattern creates an object that controls access to another object. In Shaaha, ProxyModule controls access to the real backend (e.g. numpy). The user never touches numpy directly — they only touch the proxy. This is what allows Shaaha to switch backends transparently.

Strategy Pattern — Explained

The Strategy pattern defines a family of algorithms (backends), encapsulates each one, and makes them interchangeable. The Router is Shaaha's strategy selector — it picks the right 'algorithm' (backend) based on the current context (hardware + installed packages). The user code does not change when the strategy changes.

Registry Pattern — Explained

The Registry pattern maintains a central dictionary mapping names to objects or classes. In Shaaha, REGISTRY maps domain names to lists of Backend objects. This is extensible by design — adding a new backend is just adding one entry to the dict. This is why community contributions are easy.

Chapter 7 — How Routing Works (Step by Step)

This is the complete journey from 'import shaaha.math' to 'numpy.array([1,2,3])':

#	Action	Code Location
1	User writes: import shaaha.math	User's code
2	Python checks sys.meta_path[0] — finds ShaahafFinder	importer.py
3	find_spec('shaaha.math') is called — returns ModuleSpec	importer.py
4	Python calls ShaahaaLoader.exec_module(module)	importer.py
5	ProxyModule('math', Router) is created and inserted	proxy.py
6	ProxyModule is cached in sys.modules['shaaha.math']	Python internals
7	User writes: sm.array([1,2,3])	User's code
8	ProxyModule.__getattr__('array') is triggered	proxy.py
9	ProxyModule._resolve() is called for the first time	proxy.py
10	Router.resolve('math') is called	router.py
11	Environment is checked: CUDA=False, numpy installed=True	environment.py
12	Registry returns all math backends sorted by score	registry.py
13	numpy wins (score 70, only available backend)	registry.py
14	import numpy is executed	router.py
15	numpy module is cached in Router._cache['math']	router.py
16	getattr(numpy, 'array') is returned to user	proxy.py
17	numpy.array([1,2,3]) executes normally	numpy

Chapter 8 — Supported Domains & Backends

The complete registry of all domains and their backends in Shaaha v1.0.0:

Domain	import shaaha.X	Best Backend	Fallbacks	Use Case
math	shaaha.math	cupy (GPU)	jax → torch → numpy	Arrays, linear algebra
dataframe	shaaha.dataframe	polars	modin → pandas → dask	Tabular data
ml	shaaha.ml	torch	xgboost → lightgbm → sklearn	Machine learning
image	shaaha.image	cv2	PIL/pillow → skimage	Image processing
nlp	shaaha.nlp	transformers	spacy → nltk	Text processing
viz	shaaha.viz	plotly	altair → seaborn → matplotlib	Visualisation
stats	shaaha.stats	statsmodels	scipy → sklearn	Statistics
http	shaaha.http	httpx	requests → urllib3	HTTP requests
json	shaaha.json	orjson	ujson → json (stdlib)	JSON parsing

Chapter 9 — What is the AI Element in Shaaha?

Yes, Shaaha contains AI — specifically **Context-Aware Intelligent Dispatch**. Here is exactly what that means and how to explain it in your thesis:

What Makes It 'Intelligent'

Traditional software uses fixed rules: 'always use numpy'. Shaaha uses **context-aware decision making** — its routing decision changes based on the current state of the environment. This is a form of adaptive intelligence.

Context Factor	How Shaaha Uses It	AI Concept
CUDA GPU detected	Boosts weight of cupy/torch by +20	Hardware-aware adaptation
File size (future)	Routes to polars vs pandas	Data-scale reasoning
Installed packages	Filters unavailable backends	Constraint satisfaction
User preference	configure() overrides routing	User-guided learning
Fallback chain	Tries next best if top fails	Graceful degradation

For your MSc thesis, describe the routing algorithm as a **Weighted Constraint Satisfaction Problem (CSP)**: given a set of backends (variables), constraints (hardware requirements, availability), and weights (performance scores), find the optimal assignment. This is textbook AI.

Chapter 10 — How to Extend Shaaha

Adding a new backend to Shaaha takes exactly 5 lines. Here is how:

Adding a New Backend to an Existing Domain

Open `src/shaaha/registry.py` and add one Backend entry:

```
from shaaha.registry import Backend, REGISTRY # Add vaex as a new dataframe backend
REGISTRY['dataframe'].append( Backend( name='vaex', import_name='vaex', weight=88,
tags=['large_data', 'out-of-core'] ) )
```

Adding a Completely New Domain

```
REGISTRY['audio'] = [ Backend('librosa', 'librosa', weight=90, tags=['analysis']),
Backend('soundfile', 'soundfile', weight=70, tags=['io']), ] # Now users can do: #
import shaaha.audio as audio # audio.load('song.wav')
```

Adding a Wrapper for Non-Standard APIs

Create `src/shaaha/wrappers/audio_librosa.py` with standardised function names, then reference it in the Backend definition:

```
Backend('librosa', 'librosa', weight=90, adapter='audio_librosa')
```

Chapter 11 — MSc Thesis Key Points

Here is exactly what to write in each section of your thesis:

Abstract (3 sentences)

'This thesis presents Shaaha, an open-source Python library implementing an Intelligent Meta-Dispatcher using PEP 302 import hooks. Shaaha automatically selects the optimal computational backend (numpy/cupy, pandas/polars, sklearn/torch) based on runtime hardware context and dependency availability. The system reduces boilerplate code, eliminates dependency hell, and improves computational efficiency through context-aware weighted dispatch.'

Technical Contribution Section

- Novel application of PEP 302 MetaPathFinder for library abstraction
- Context-aware weighted scoring algorithm for backend selection
- Lazy proxy module system with zero startup overhead
- Extensible registry architecture for community contributions
- Production-ready open-source release on PyPI

Academic References to Cite

- PEP 302 — New Import Hooks (van Rossum, 2003)
- PEP 451 — A ModuleSpec Type for the Import System (Snow, 2013)
- Gang of Four — Design Patterns: Elements of Reusable Object-Oriented Software
- Python Documentation — The Import System (docs.python.org)
- Gamma et al. — Proxy Pattern (Design Patterns book, 1994)

What Makes It Novel

Similar tools exist (anyio, array-api-compat) but none combine: (1) PEP 302 import hook interception, (2) hardware-aware weighted routing, (3) lazy proxy delegation, and (4) a universal domain registry — all in a zero-dependency package.

Chapter 12 — Quick Reference Card

Key Python Concepts Used

Concept	What It Is	Where Used
<code>sys.meta_path</code>	List of Finder objects Python checks on every import	<code>__init__.py</code>
<code>importlib.abc.MetaPathFinder</code>	Base class for custom import finders	<code>importer.py</code>
<code>importlib.abc.Loader</code>	Base class for custom module loaders	<code>importer.py</code>
<code>importlib.machinery.ModuleSpec</code>	Metadata object describing a module	<code>importer.py</code>
<code>types.ModuleType</code>	Base class for all Python modules	<code>proxy.py</code>
<code>__getattr__</code>	Called when normal attribute lookup fails	<code>proxy.py</code>
<code>importlib.util.find_spec()</code>	Checks if a package is importable without importing	<code>itenvironment.py</code>
<code>importlib.invalidate_caches()</code>	Refreshes the module finder after pip install	<code>installer.py</code>
<code>dataclasses.dataclass</code>	Auto-generates <code>__init__</code> , <code>__repr__</code> etc.	<code>environment.py</code> , <code>registry.py</code>
<code>subprocess.run()</code>	Runs external commands (pip install)	<code>installer.py</code>
<code>sys.executable</code>	Path to the current Python interpreter	<code>installer.py</code>

Essential Commands

Command	What It Does
<code>pip install shaaha</code>	Install Shaaha from PyPI
<code>pip install shaaha[default]</code>	Install with numpy, pandas, pillow, sklearn, matplotlib
<code>pip install shaaha[gpu]</code>	Install GPU stack (torch, cupy)
<code>python -m build</code>	Build the distribution packages (wheel + tarball)
<code>python -m twine upload dist/*</code>	Upload to PyPI
<code>python -m pytest tests/ -v</code>	Run all 24 tests
<code>shaaha.status()</code>	Show current environment and selected backends
<code>shaaha.available_backends('math')</code>	List installed backends for math domain
<code>shaaha.configure(prefer_gpu=False)</code>	Force CPU-only backend selection
<code>shaaha.configure(log_level='DEBUG')</code>	See routing decisions in real time

Shaaha v1.0.0 — pip install shaaha
pypi.org/project/shaaha
Built with PEP 302 | MIT License | MSc Thesis Project
